Practical 10

Aim: Scalable K-Means Clustering on Customer Segmentation Data using Apache Spark MLlib (Parallel K-Means)

Problem definition: You are a Data Scientist at RetailEdge Analytics. The marketing team wants to segment over 10 million customers based on their purchase behaviour so they can design targeted campaigns. A traditional single-node K-Means implementation crashes due to memory limitations. Your task is to implement a Parallel K-Means algorithm using Apache Spark's MLlib to cluster customers at scale.

Observation / Output:

1.Start HDFS

```
C:\SEM5\BDA\Prac10>jps
19860 DataNode
7508 NameNode
1944 ResourceManager
19560 Jps
12956 NodeManager
```

2.Make folder in HDFS

```
drwxr-xr-x   - satya supergroup          0 2026-08-22 14:36 /bda
-rw-r--r--   1 satya supergroup         15 2026-07-15 22:15 /hello.txt
drwxr-xr-x   - satya supergroup          0 2026-07-15 22:16 /input
drwxr-xr-x   - satya supergroup          0 2026-10-08 12:15 /practical10
drwxr-xr-x   - satya supergroup          0 2026-08-13 21:03 /practical3
drwxr-xr-x   - satya supergroup          0 2026-08-14 15:48 /practical4
drwxr-xr-x   - satya supergroup          0 2026-07-15 21:32 /test
drwxr-xr-x   - satya supergroup          0 2026-07-15 22:16 /tmp
```

3.upload csv file in hdfs

```
C:\SEM5\BDA\Prac10>hdfs dfs -put -f customer_transactions.csv /practical10/input/
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.apache.hadoop.util.Nati
file:/C:/hadoop-3.5.0/hadoop-3.5.0/share/hadoop/common/hadoop-common-3.5.0.jar)
```

```
C:\SEM5\BDA\Prac10>hdfs dfs -ls /practical10/input
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.apache.hadoop.util.NativeCodeLoader in an u
file:/C:/hadoop-3.5.0/hadoop-3.5.0/share/hadoop/common/hadoop-common-3.5.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::arrayBaseOffset has been called by org.apache.hadoop.thirdparty.protobuf.UnsafeU
ssor (file:/C:/hadoop-3.5.0/hadoop-3.5.0/share/hadoop/common/lib/hadoop-shaded-protobuf_3_25-1.5.0.jar)
WARNING: Please consider reporting this to the maintainers of class org.apache.hadoop.thirdparty.protobuf
oryAccessor
WARNING: sun.misc.Unsafe::arrayBaseOffset will be removed in a future release
Found 1 items
-rw-r--r--   1 satya supergroup        549 2026-10-08 12:16 /practical10/input/customer_transactions.csv
```

4. Load and preprocess the large-scale customer transaction dataset from HDFS.

```
>>> df = spark.read.csv(
...     "hdfs://localhost:9000/practical10/input/customer_transactions.csv"\


...     header=True,
... )   inferSchema=True
>>> df.show()
+----------+---------------+------+
|CustomerID|TransactionDate|Amount|
+----------+---------------+------+
|      C001|     2026-09-01|   500|
|      C001|     2026-09-10|   800|
|      C001|     2026-09-20|  1200|
|      C002|     2026-08-15|   300|
```

5.Extract features: total spend, purchase frequency, average basket size, and recency.

```
>>> total_spend = df.groupBy("CustomerID").agg(
...     sum("Amount").alias("Total_Spend")
... )
>>> total_spend.show()
+----------+-----------+
|CustomerID|Total_Spend|
+----------+-----------+
|      C006|       7500|
|      C010|      15000|
|      C007|        600|
|      C003|       7500|
|      C004|        200|
|      C009|       1300|
|      C008|       4000|
|      C005|       2700|
|      C001|       2500|
|      C002|        750|
+----------+-----------+
```

```
>>> frequency = df.groupBy("CustomerID").agg(
...     count("*").alias("Purchase_Frequency")
... )
>>> frequency.show()
+----------+------------------+
|CustomerID|Purchase_Frequency|
+----------+------------------+
|      C006|                 2|
|      C010|                 3|
|      C007|                 3|
|      C003|                 3|
|      C004|                 1|
|      C009|                 2|
|      C008|                 2|
|      C005|                 3|
|      C001|                 3|
|      C002|                 2|
+----------+------------------+
```

```
>>> avg_basket = df.groupBy("CustomerID").agg(
...     avg("Amount").alias("Average_Basket_Size")
... )
>>> avg_basket.show()
+----------+------------------+
|CustomerID|Average_Basket_Size|
+----------+------------------+
|      C006|            3750.0|
|      C010|            5000.0|
|      C007|             200.0|
|      C003|            2500.0|
|      C004|             200.0|
|      C009|             650.0|
|      C008|            2000.0|
|      C005|             900.0|
|      C001| 833.3333333333334|
|      C002|             375.0|
+----------+------------------+
```

```
>>> reference_date = df.select(
...     max("TransactionDate")
... ).first()[0]
>>> print("Reference Date:", reference_date)
Reference Date: 2026-09-20
>>>
```

```
... )
>>> recency.show()
+----------+-------------+-------+
|CustomerID|Last_Purchase|Recency|
+----------+-------------+-------+
|      C006|   2026-08-15|     36|
|      C010|   2026-08-01|     50|
|      C007|   2026-09-05|     15|
|      C003|   2026-09-15|      5|
|      C004|   2026-09-18|      2|
|      C009|   2026-09-20|      0|
|      C008|   2026-08-10|     41|
|      C005|   2026-09-10|     10|
|      C001|   2026-09-20|      0|
|      C002|   2026-09-05|     15|
+----------+-------------+-------+
```

6. Apply feature scaling (StandardScaler) to normalize the features.

```
+----------+-----------------------------------+
|CustomerID|features                           |
+----------+-----------------------------------+
|C006      |[7500.0,2.0,3750.0,36.0]           |
|C010      |[15000.0,3.0,5000.0,50.0]          |
|C007      |[600.0,3.0,200.0,15.0]             |
|C003      |[7500.0,3.0,2500.0,5.0]            |
|C004      |[200.0,1.0,200.0,2.0]              |
|C009      |[1300.0,2.0,650.0,0.0]             |
|C008      |[4000.0,2.0,2000.0,41.0]           |
|C005      |[2700.0,3.0,900.0,10.0]            |
|C001      |[2500.0,3.0,833.3333333333334,0.0]|
|C002      |[750.0,2.0,375.0,15.0]             |
+----------+-----------------------------------+
```

```
+----------+--------------------------------------------------------------
------------------+
|CustomerID|scaledFeatures
                    |
+----------+--------------------------------------------------------------
------------------+
|C006      |[0.7117734761265 67,-0.5720775535473559,1.2773909036516973,1.
75776948482]       |
|C010      |[2.33189519720373,0.8581163303210326,2.034438061090475,1.77
509957264]         |
|C007      |[-0.7787385072644232,0.8581163303210326,-0.8726230234744311,
3083558412238 49]  |
|C003      |[0.7117734761265 67,0.8581163303210326,0.5203437462129197,-0.
838512989882]      |
|C004      |[-0.8651449990552051,-2.0022714374157444,-0.872623023474431
8395283314519691] |
|C009      |[-0.6275271466305545,-0.5720775535473559,-0.600086046796471
9485579848872898] |
|C008      |[-0.0442833270427757 94,-0.5720775535473559,0.217524883237408
.2865499105367837]|
|C005      |[-0.3251044253628174,0.8581163303210326,-0.4486766153087156,
```

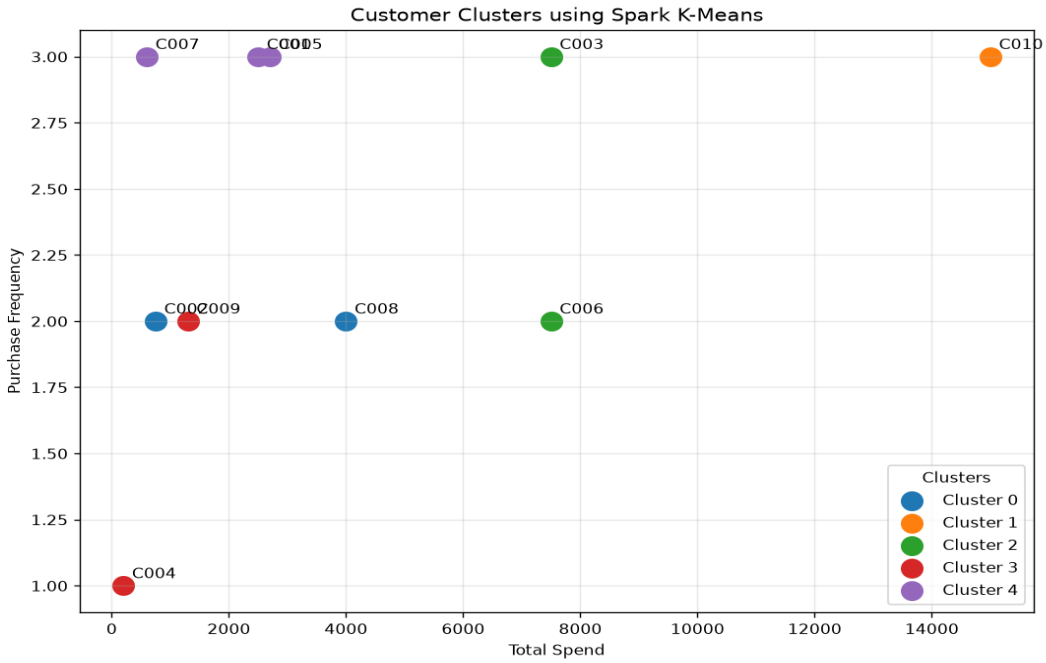
7. Implement Spark MLlib's KMeans with k=5 clusters.

```
>>> from pyspark.ml.clustering import KMeans
>>> kmeans = KMeans(
...     k=5,
...     seed=42,
...     featuresCol="scaledFeatures",
...     predictionCol="prediction"
... )
>>> model = kmeans.fit(scaled_data)
netlib-blas: JNI_OnLoad: dlopen(libopenblas.dll) failed: LoadLibrary failed
>>> print(model)
KMeansModel: uid=KMeans_b664d1ede3e1, k=5, distanceMeasure=euclidean, numFeatures=4
>>> predictions = model.transform(scaled_data)
>>> predictions.select(
...     "CustomerID",
...     "Total_Spend",
...     "Purchase_Frequency",
...     "Average_Basket_Size",
...     "Recency",
...     "prediction"
... ).orderBy("CustomerID").show()
+----------+-----------+------------------+------------------+-------+----------+
|CustomerID|Total_Spend|Purchase_Frequency|Average_Basket_Size|Recency|prediction|
+----------+-----------+------------------+------------------+-------+----------+
|      C001|       2500|                 3|  833.3333333333334|      0|         4|
|      C002|        750|                 2|             375.0|     15|         0|
|      C003|       7500|                 3|            2500.0|      5|         2|
|      C004|        200|                 1|             200.0|      2|         3|
|      C005|       2700|                 3|             900.0|     10|         4|
|      C006|       7500|                 2|            3750.0|     36|         2|
|      C007|        600|                 3|             200.0|     15|         4|
|      C008|       4000|                 2|            2000.0|     41|         0|
|      C009|       1300|                 2|             650.0|      0|         3|
|      C010|      15000|                 3|            5000.0|     50|         1|
+----------+-----------+------------------+------------------+-------+----------+
```

8. Evaluate clustering quality using the Within Set Sum of Squared Errors(WSSSE).

```
>>> wssse = model.summary.trainingCost
>>> print("WSSSE:", wssse)
WSSSE: 6.148107569183518
```

9. Visualize cluster centers and interpret the customer segments.



10. Compareperformance: single-node sklearn K-Means vs Spark Parallel K-Means.

```
C:\SEM5\BDA\Prac10>python sklearn_kmeans.py
----- SKLEARN K-MEANS -----
Execution time: 1.9661340713500977 seconds
WSSSE: 3.6556480999158882
```

Key Questions /Analysis:

Q1. Why does traditional K-Means fail at large scale, and how does Spark's parallel K-Means overcome this?

Traditional K-Means runs mainly on a single machine, so processing a very large dataset can require high memory and computation time. It may become slow or unable to handle datasets that do not fit into memory. Spark K-Means distributes the data across multiple partitions and performs computations in parallel. This makes it more suitable for large-scale datasets and reduces processing time.

Q2. What is the significance of the WSSSE metric? How do you use the Elbow Method to select optimal k?

WSSSE (Within Set Sum of Squared Errors) measures how close the data points are to their assigned cluster centers. A lower WSSSE generally means that the clusters are more compact. In the Elbow Method, K-Means is run with different values of k, and the WSSSE is plotted against k. The point where the decrease in WSSSE starts becoming smaller is called the elbow, and that value is selected as the suitable k.

Q3. How does data partitioning in Spark affect convergence speed of K-Means?

Spark divides the dataset into partitions and processes these partitions in parallel. Properly balanced partitions allow the available resources to work efficiently and can improve the execution speed of K-Means iterations. Poorly balanced partitions can cause some tasks to take longer than others, increasing the overall execution time.

Q4. What customer behaviour pattern does each cluster represent? How can marketing strategies differ per segment?

Each cluster represents customers with similar spending and purchasing behaviour. Based on the cluster profile, segments can represent:

- High-value active customers: High spending and frequent purchases. Offer loyalty rewards and premium products.
- Occasional high-value customers: High spending but fewer purchases. Use personalized offers to increase purchase frequency.
- Frequent low-value customers: Frequent purchases with lower spending. Use bundle offers and cross-selling.
- Inactive or at-risk customers: Low activity or high recency. Use discounts and re-engagement campaigns.

Q5. How does feature scaling impact the clustering result? What happens if features are not normalized?

Feature scaling puts all features on a comparable scale before calculating distances. In this practical, Total Spend, Purchase Frequency, Average Basket Size, and Recency have different ranges. Standardization prevents a large-valued feature such as Total Spend from dominating the distance calculation. Without scaling, K-Means may give more importance to features with larger numerical values, resulting in less meaningful clusters.

.

Conclusion:

In this practical, Spark MLlib K-Means was used for scalable customer segmentation. The customer features were standardized and five clusters were generated using K-Means. The WSSSE metric was used to evaluate cluster compactness, while cluster profiles and visualization helped understand customer behaviour. Spark's partition-based parallel processing makes K-Means more suitable for large datasets than traditional single-machine K-Means. The resulting customer segments can also help businesses apply different marketing strategies to different types of customers.